

Excellent 👍 — let's go deep into **DMVs (Dynamic Management Views)** in SQL Server — one of the **most powerful tools for performance monitoring, troubleshooting, and tuning**.

You'll use DMVs constantly as a DBA or backend developer to diagnose blocking, slow queries, memory pressure, index usage, and more.

What are DMVs?

DMVs (Dynamic Management Views) and **DMFs (Dynamic Management Functions)** are **system views and functions** that return real-time information about the SQL Server internals — including:

- Server health
- Database performance
- Query execution statistics
- Index usage
- Locking & blocking
- Memory usage

👉 Think of them as **SQL Server's diagnostic sensors**.

Syntax / Naming Convention

All DMVs and DMFs start with:

sys.dm_

Common categories include:

- sys.dm_exec_* → execution & query stats
- sys.dm_db_* → database & index info
- sys.dm_os_* → operating system & memory
- sys.dm_tran_* → transactions & locks
- sys.dm_io_* → I/O statistics

Common DMV Examples (Category-wise)

Let's see some **real-world useful DMV queries** you'll use often 🙌

◆ 1. Check Currently Running Queries

```
SELECT
    r.session_id,
    r.status,
    r.cpu_time,
    r.total_elapsed_time,
    SUBSTRING(t.text, (r.statement_start_offset/2)+1,
        ((CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(t.text)
            ELSE r.statement_end_offset END - r.statement_start_offset)/2)+1) AS running_query,
    DB_NAME(r.database_id) AS database_name
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id <> @@SPID;
```

 **Use:** See what SQL statements are currently executing.

◆ 2. Find Blocking / Waits

```
SELECT
    blocking_session_id AS blocker,
    session_id AS blocked,
    wait_type,
```

```
wait_time,  
wait_resource  
FROM sys.dm_exec_requests  
WHERE blocking_session_id IS NOT NULL;
```

 **Use:** Identify which session is blocking another.

◆ 3. Top CPU-Consuming Queries

```
SELECT TOP 10  
  
total_worker_time/1000 AS total_cpu_ms,  
execution_count,  
total_worker_time/execution_count/1000 AS avg_cpu_ms,  
SUBSTRING(qt.text, qs.statement_start_offset/2,  
  (CASE WHEN qs.statement_end_offset = -1  
    THEN LEN(CONVERT(NVARCHAR(MAX), qt.text)) * 2  
    ELSE qs.statement_end_offset END - qs.statement_start_offset)/2) AS query_text  
FROM sys.dm_exec_query_stats AS qs  
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS qt  
ORDER BY total_worker_time DESC;
```

 **Use:** Find the top CPU-heavy queries.

◆ 4. Index Usage Statistics

```
SELECT  
  
OBJECT_NAME(s.[object_id]) AS table_name,  
i.name AS index_name,  
s.user_seeks, s.user_scans, s.user_lookups, s.user_updates  
FROM sys.dm_db_index_usage_stats AS s
```

JOIN sys.indexes AS i

ON s.[object_id] = i.[object_id]

AND s.index_id = i.index_id

WHERE OBJECTPROPERTY(s.[object_id], 'IsUserTable') = 1;

 **Use:** Identify which indexes are actually used (or never used → can drop).

◆ 5. Missing Indexes (SQL's Recommendation)

SELECT

DB_NAME(database_id) AS database_name,

OBJECT_NAME(object_id, database_id) AS table_name,

equality_columns,

inequality_columns,

included_columns,

avg_total_user_cost,

avg_user_impact,

last_user_seek

FROM sys.dm_db_missing_index_details;

 **Use:** See which indexes SQL Server recommends creating.

◆ 6. Wait Statistics

SELECT TOP 15

wait_type,

wait_time_ms / 1000.0 AS wait_time_sec,

signal_wait_time_ms / 1000.0 AS signal_wait_time_sec,

waiting_tasks_count

FROM sys.dm_os_wait_stats

WHERE wait_type NOT LIKE 'SLEEP%'

ORDER BY wait_time_ms DESC;

 **Use:** Identify what the server is waiting on (I/O, locks, CPU, etc.).

◆ 7. Memory Usage

SELECT

type,

SUM(virtual_memory_committed_kb)/1024 AS VM_MB,

SUM(virtual_memory_reserved_kb)/1024 AS VR_MB

FROM sys.dm_os_memory_clerks

GROUP BY type

ORDER BY VM_MB DESC;

 **Use:** Check which components use the most memory.

◆ 8. Database I/O Stats

SELECT

DB_NAME(database_id) AS database_name,

file_id,

io_stall_read_ms,

num_of_reads,

io_stall_write_ms,

num_of_writes

FROM sys.dm_io_virtual_file_stats(NULL, NULL);

 **Use:** Measure read/write latency per database file.

◆ 9. TempDB Usage

```
SELECT  
  
    DB_NAME(database_id) AS database_name,  
  
    SUM(internal_object_reserved_page_count)*8 AS InternalObjectsKB,  
  
    SUM(user_object_reserved_page_count)*8 AS UserObjectsKB  
  
FROM sys.dm_db_task_space_usage  
  
GROUP BY database_id;
```

 **Use:** See which database or query is consuming TempDB space.

◆ 10. Active Transactions

```
SELECT  
  
    transaction_id,  
  
    name,  
  
    transaction_begin_time,  
  
    transaction_state,  
  
    transaction_type  
  
FROM sys.dm_tran_active_transactions;
```

 **Use:** Check all open transactions and their states.

DMV Category Summary

Category	Prefix	Example	Description
Execution	sys.dm_exec_*	sys.dm_exec_requests	Current queries, plans, performance
Database	sys.dm_db_*	sys.dm_db_index_usage_stats	Indexes, missing indexes
OS	sys.dm_os_*	sys.dm_os_wait_stats	CPU, memory, waits

Category	Prefix	Example	Description
I/O	sys.dm_io_*	sys.dm_io_virtual_file_stats	Disk activity
Transactions	sys.dm_tran_*	sys.dm_tran_locks	Locks, blocking, active transactions
Query Store	sys.query_store_*	sys.query_store_query	Query store history

⚠ DMV Important Notes

- DMVs show **data since the last SQL Server restart** (resets after restart).
 - You can query them anytime (read-only).
 - Combine multiple DMVs for deep insights (e.g., execution + query plan).
 - Great for **performance tuning, monitoring, and troubleshooting**.
-

✅ Real-World Use

Scenario	DMV to Use
Find slow queries	sys.dm_exec_query_stats
Detect blocking	sys.dm_exec_requests, sys.dm_tran_locks
Identify unused indexes	sys.dm_db_index_usage_stats
Check missing indexes	sys.dm_db_missing_index_details
Diagnose waits	sys.dm_os_wait_stats
Check TempDB pressure	sys.dm_db_task_space_usage
Monitor I/O	sys.dm_io_virtual_file_stats
View memory consumers	sys.dm_os_memory_clerks